

南開大學
Nankai University

南开大学

计 算 机 学 院

编译原理实验报告

了解编译器：LLVM IR 编程与 RISC-V 汇编编程

马禹翔 2410933

胡家华 2410935

指导教师：王刚

2026 年 10 月 10 日

摘要

本实验围绕编译器完整处理流程，将同一 SysY 程序分别表示为 LLVM IR 和 RISC-V 汇编，并链接课程提供的 SysY 运行库执行。实验以斐波那契程序对齐小组两人的成果，另设计包含整数运算、短路逻辑、作用域、循环控制和函数调用的补充程序，并扩展一维数组与单精度运算。采用 Clang/LLVM 生成 RV64GC、LP64D 静态 ELF，在 QEMU 中执行，以源程序编译结果和独立 Python 算法共同核验。共完成 110 组输入、570 次执行，标准输出及退出状态全部符合预期。实验还比较了 `-O0`、`-O2` 与调试选项产生的中间文件，说明目标设置、调用约定和运行库兼容性对跨架构整合的影响。

目录

一、 引言 **1**

二、 实验环境与编译处理流程 **1**

（一） 实验环境 1

（二） 语言处理系统的各阶段 1

三、 示例程序与语言特性覆盖 **2**

（一） 斐波那契：小组共同的语义基准 2

（二） 补充例的设计 2

四、 LLVM IR 成果整合 **3**

（一） 原始成果 3

（二） 由 x86 目标移植到 RISC-V 3

（三） 补充程序的 IR 设计 4

五、 RISC-V 汇编实现 **4**

（一） 调用约定与数据宽度 4

（二） 斐波那契循环 4

（三） 补充汇编的关键细节 5

六、 汇编、链接与运行验证 **5**

（一） 链接课程 SysY 运行库 5

（二） 测试方法与结果 6

七、 优化选项与结果讨论 **6**

（一） 优化和调试选项的影响 6

（二） 跨架构整合经验 7

八、 进阶要求：MLIR 与 AscendNPU IR **7**

（一） 多级 IR 与语义逐层下降 7

（二） VecAdd 的逐层 Lowering 分析 8

（三） Pass 流水线、工具链与验证范围 9

九、 结论与复现方法 **9**

小组分工

成员	分工及本报告采用的成果
马禹翔（2410933）	RISC-V 汇编分工：斐波那契汇编、语言特性补充例、运行库链接、交叉验证与报告整合。
胡家华（2410935）	LLVM 分工：原始斐波那契 C/SysY、手写 LLVM IR、LLVM 观察记录及 MLIR 文档阅读探索。

合作仓库基准为 hjhuiuaa/Compiler 的“预备工作”分支.

一、 引言

高级语言中的变量、表达式和控制流，需要逐步转化为机器指令与可执行文件。本实验的重点是验证这些表示之间的对应关系，而不止于生成一份汇编文本。我们将任务分成两条路径：一条保存预处理、抽象语法树、LLVM IR、目标汇编和 ELF 的阶段证据；另一条验证手写 LLVM IR 与手写 RISC-V 汇编的可观察行为。两条路径最终使用同一目标架构、同一运行库和相同输入汇合。

课程基础要求涉及整数类型、常量与变量、赋值、条件分支、循环、函数、作用域、`break`/`continue`、逻辑运算及 SysY 运行库连接。指导书给出的斐波那契程序适合观察循环，却不能单独覆盖所有特性。因此，本实验保留斐波那契作为小组共同语义基准，同时加入语言特性补充例.

二、 实验环境与编译处理流程

（一） 实验环境

表 1 整合后重新验证的实验环境

项目	实际配置
宿主与工具环境	Windows；独立 WSL2 Alpine Linux 3.22
编译器与链接器	Alpine Clang 20.1.8；LLD 20.1.8
执行器	qemu-riscv64 10.0.0
目标与 ABI	`riscv64-linux-musl`；`-march=rv64gc`；`-mabi=lp64d`
标准库与 SysY 库	RISC-V musl 1.2.5-r12；课程 `sylib.c` 重新编译得到的静态库

（二） 语言处理系统的各阶段

Clang 命令本身是驱动程序，负责调度前端、优化器、目标代码生成器、汇编器和链接器。不同停止选项可以保存各阶段输出 [3]。

表 2 编译阶段、输出及本次观察内容

阶段	保存的输出	主要观察
预处理	`fib.i`、`fib.tokens.txt`	展开头文件和宏，处理条件编译与行标记，得到记号序列。
语法与语义分析	`fib.ast.txt`	观察 `VarDecl`、`WhileStmt`、`CallExpr` 及表达式类型。
IR 生成与优化	`fib.O0.ll`、`fib.O2.ll`	观察栈槽、`load/store`、比较、分支和 SSA 合并。
目标代码生成	`fib.O0.s`、`fib.O2.s`	观察指令选择、寄存器分配、栈帧与函数调用。
汇编	`*.o`、`object.txt`	将助记符编码为指令，建立节、符号与待处理重定位。
链接	ELF、`*.map`	合并目标文件与库，解析外部符号并布局最终地址。

核心命令如下；其中 `$TARGET` 表示目标三元组、ABI 和 sysroot 参数，完整命令及退出码保存在 `build/commands.json`。

Listing 1 编译流程中的阶段命令

```
clang $TARGET -E fib.c -o fib.i
clang $TARGET -fsyntax-only -Xclang -ast-dump fib.c
clang $TARGET -O0 -S -emit-llvm fib.c -o fib.ll
clang $TARGET -O0 -S fib.c -o fib.s
clang $TARGET -c fib.s -o fib.o
```

观察 C 程序时使用 `stdio` 接口；手写 IR 和汇编使用 `getint`、`putint`、`putch`。两者的算法和标准输出相同。

三、 示例程序与语言特性覆盖

（一） 斐波那契：小组共同的语义基准

沿用合作仓库中的 `fib.sy`：先输出 0 和 1，令 $i = 1$；当 $i < n$ 时执行 $t = b$、$b = a + b$、输出 b、$a = t$、$i = i + 1$。输入 5 时得到 0、1、1、2、3、5，共六行。$n \leq 1$ 时仍然输出前两项，这是指导书程序本身的行为，因此没有擅自改成“输出 n 项”。测试限制到 $n \leq 46$，因为 $F(46) = 1836311903$ 仍在有符号 32 位整数范围内，而 $F(47)$ 会溢出。

（二） 补充例的设计

`examples/features.sy` 读取整数 x、d，测试约定二者均位于 $[-100, 100]$。程序依次输出加、减、乘、除、余、符号分类、逻辑结果、副作用计数、数组归约、浮点转整数和作用域观察值。$d = 0$ 时显式避开除法与取模。

表 3 补充例对语言特性的覆盖

特性	示例位置或验证方式
常量、变量、初始化、赋值	`WIDTH`、`calls`，局部 x/d，`reduce` 中的 i/sum 更新
算术与单目运算	主函数前五项结果及 `emit(-x)`
六种关系运算	符号分类、循环条件、短路条件与逻辑非组合
逻辑与、逻辑或、逻辑非	除数保护、`touch` 副作用与 `!(x <= d)`
条件、循环和控制转移	`if/else`、`while`、`break`、`continue`
函数与返回	`touch`、`reduce`、`blend`、`emit`
嵌套作用域	内层 `int x=d+1`，离开语句块后再次输出外层 x
运行库输入输出	`getint` 输入，`putint/putch` 输出
额外覆盖	一维整数数组与数组参数；`float` 运算及隐式转换

数组初始化为 `{x,-2,d,0,9}`。`reduce` 先检查零，再跳过负数，因此尾部 9 永远不累加，可以同时检验 `break` 和 `continue`。浮点表达式为

$$z = \frac{(x + 0.5) \times 2 - d}{2},$$

转成整数时向零截断。测试范围内的中间半整数都可以精确表示，因此这里不需要浮点误差容限。

SysY 是 C 的子集，构建脚本用 `-x c` 和单独的接口声明编译这些 `.sy` 文件作为参考实现；这不代表 Clang 内建了 SysY 前端。本实验也没有声称覆盖多维数组、张量扩展或实现完整 SysY 编译器。

四、 LLVM IR 成果整合

（一） 原始成果

`fib.ll` 是 Clang 15 自动生成的参考 IR，`fib-sysy.ll` 是将输入输出替换成 SysY 运行库后的手写版本。原始 IR 将 a, b, i, t, n 放在 `i32` 栈槽中；赋值对应 `store`，读取变量对应 `load`；循环条件使用 `icmp slt`，随后由 `br` 选择循环体或出口。输入使用 `call i32 @getint()`，每个整数的输出由 `putint` 与 `putch(10)` 组成。

记录到数字 SSA 名称不连续导致 `llvm-as` 报错的问题。数字结果名与数字基本块标签共享编号顺序，`void` 调用不占结果编号；移除原先的 `printf` 调用以后，需要同步调整后续数字名称。补充 IR 改用语义化名称，以降低维护负担。LLVM 的 `ptr` 为不透明指针，访问数据的类型由 `load/store` 等指令指定 [4]。

（二） 由 x86 目标移植到 RISC-V

原始文件带有 `x86_64-pc-linux-gnu` 目标三元组、x86 数据布局和 `target-cpu/target-features/tune-cpu` 属性，不能只更改文件扩展名就当作 RISC-V 输入。本次保留原文件，由 `scripts/port_ir.py` 生成 `ir/fib-riscv.ll`：替换目标三元组，移除旧数据布局与

x86 专属属性，并删除不再调用的 stdio 声明。Clang 编译时再以显式目标参数确定实际 Linux musl 数据布局。

斐波那契的基本块结构和 i32 语义没有改变。llvm-as 将文本 IR 转成 bitcode，opt -passes=verify 检查结构，之后经 Clang 后端生成 RV64 目标文件。该程序没有指针与整数互转等依赖原数据布局的操作，适合这样移植；这一经验不能无条件推广到任意 LLVM IR。

（三） 补充程序的 IR 设计

ir/features-riscv.ll 是新增的手写实现。reduce 通过 phi 表示循环携带的 i 与 sum，getelementptr 按 i32 元素寻址；主函数将短路求值拆成多个基本块，确保 $d = 0$ 时不会执行 x/d；浮点部分使用 fadd/fmul/fsub/fdiv，再由 fptosi 转换为整数。

Listing 2 逻辑与的短路控制流

```
and.left:
  br i1 %nonzero, label %and.right, label %and.false
and.right:
  %q2 = sdiv i32 %x, %d
  %gt = icmp sgt i32 %q2, 1
  br i1 %gt, label %and.true, label %and.false
```

不能提前计算带除法的右操作数再执行普通按位与：即使最终布尔值为假，提前求值仍然会改变源程序行为。同理，逻辑或右侧的 touch 只能在左侧为假时调用。

五、 RISC-V 汇编实现

（一） 调用约定与数据宽度

本实验采用 RV64GC/LP64D。整数参数和返回值通过 a0--a7 传递，blend 的两个浮点参数使用 fa0、fa1。跨函数调用后仍需存活的整数放在 s 类寄存器中，函数在入口保存、出口恢复；ra 也必须保存，因为 call 会覆盖它。所有调用处的 sp 均保持 16 字节对齐，符合 RISC-V psABI [5]。

表 4 手写汇编的寄存器与栈帧安排

程序或函数	存储分配
fib 的 main	s0=a、s1=b、s2=i、s3=n；48 字节栈帧保存 ra 和 s0--s3
features 的 main	s0=x、s1=d；64 字节栈帧；sp+0 起存放五个 i32 元素
emit	16 字节栈帧保存 ra，依次调用 putint 和 putch
reduce、touch、blend	叶函数，使用临时或参数寄存器，无需建立栈帧

RV64 的整数寄存器宽度是 64 位，但 SysY int 仍为 32 位。因此整数计算使用 addw/subw/mulw/divw/remw，数组访存使用 lw/sw；保存 ra 和 s 寄存器则使用 ld/sd。

（二） 斐波那契循环

Listing 3 手写斐波那契循环

```
.Lfib_cond:
    bge s2, s3, .Lfib_exit
    mv t0, s1
    addw s1, s0, s1
    mv s0, t0
    mv a0, s1
    call putint
    li a0, 10
    call putch
    addiw s2, s2, 1
    j .Lfib_cond
```

循环头保持不变式 `s0=F(i-1)`、`s1=F(i)`。一次更新后输出 $F(i+1)$，再递增 i，所以执行顺序与 SysY 源程序一致。源程序中的临时变量 t 放在 `t0`；在下一次函数调用前，它已转移到保存寄存器 `s0`，因而不会错误依赖调用者保存的临时寄存器。

（三） 补充汇编的关键细节

逻辑与首先用 `beqz` 检查 d，逻辑或在 $d = 0$ 时直接跳过 `touch`。数组元素地址由基址加 $i \times 4$ 得到。浮点部分使用 `flw`、`fadd.s`、`fmul.s`、`fsub.s`、`fdiv.s`；`fcvt.w.s` 明确选择 `rtz`（向零舍入）。若使用默认舍入模式，半整数用例会偏离语言规定的浮点转整数行为。内层同名变量 x 仅暂存在参数寄存器中，外层 x 始终保留于 `s0`，实现作用域隔离。

六、 汇编、链接与运行验证

（一） 链接课程 SysY 运行库

本实验使用目标 Clang 重新编译 `sylib.c`，再以 `llvm-ar` 建立 `build/libsysy_riscv.a`。程序与库使用相同的目标架构、ABI 与 RISC-V musl 标准库；没有链接宿主 x86 库，也没有用自制输入输出函数代替课程运行库。

实验采用静态链接，显式加入 musl 启动文件、`libc.a` 与目标 `compiler-rt` 内建函数库。首次链接出现 `__addtf3` 等未定义符号，原因是 libc 格式化输入输出需要四倍精度辅助运算；补入 RISC-V builtins 后解决，初次诊断保存在 `evidence/initial-link-missing-builtins.json`。

`fib-asm.o` 中的 `getint/putint/putch` 是未定义符号，调用位置带 RISC-V 重定位；链接完成后，最终符号表中出现它们的定义。`readelf` 确认最终程序为 RISC-V ELF64 静态可执行文件，ELF 标志包含 RVC 和 double-float ABI，QEMU 可以直接运行。重定位、反汇编、符号表和链接映射都保存在 `build/stages`。

（二） 测试方法与结果

表 5 差分测试与独立参考测试结果

测试集	输入与版本	实测结果
斐波那契	$-5, 0, 1, 2, 3, 5, 10, 20, 30, 46$；7 个版本	$10 \times 7 = 70$ 次通过
语言特性	49 组符号/零/边界组合和 51 组固定种子随机输入；5 个版本	$100 \times 5 = 500$ 次通过
汇总	110 组输入；570 次独立进程执行	570 通过，0 失败

五个基本版本分别为 SysY 源程序的 `-O0/-O2`、手写 IR 的 `-O0/-O2` 与手写汇编；斐波那契还执行原始 `fib.c` 的 `-O0/-O2`。每个版本都与独立 Python 算法生成的预期字符串逐字比较，并要求退出码为 0。

输入 `7 3` 时，补充程序依次输出

$$10, 4, 21, 2, 1, 1, 1, 1, 1, 1, 10, 6, -7, 4, 7.$$

输入 `7 0` 时，副作用计数为 0，说明逻辑或确实跳过 `touch`；浮点结果为 7，检验了 7.5 向零截断。负数测试用于检查有符号比较、除法和余数语义，测试预言没有直接使用 Python 的 `//` 与 `%`，因为它们对负数的规则与 C/SysY 不同。

测试将标准输出和标准错误分开保存。运行库的 `TOTAL: ...` 行属于标准错误，不是程序数值输出，也没有被用于性能评价。完整输入、预期值、实际输出和退出码见 `build/test-results.json`。有限测试证明这些用例上的一致性，但不等价于对所有输入的形式化证明。

七、 优化选项与结果讨论

（一） 优化和调试选项的影响

表 6 `fib.c` 在不同优化级别下的真实输出统计

观察指标	`-O0`	`-O2`
IR 中 `alloca` 数量	6	1
IR 中 `load` 数量	10	2
IR 中 `store` 数量	8	0
IR 中 `phi` 数量	0	3
目标文件 `text` 列字节数	247	197

`-O0` 便于将源变量对应到栈槽；`-O2` 将可提升的变量合并为 SSA 值，减少内存访问，并改变控制流及寄存器保存安排。`scanf` 需要取得 n 的地址，因此 n 的存储需求与其他局部变量不同。表中的 `text` 是 `llvm-size` 的统计口径，包含只读内容等，不能解释成 `main` 单个函数的纯指令长度；这里只做同一源码配置间的比较，不据此推导运行速度。

`fib` IR 保留了 `optnone`，所以给编译命令加入 `-O2` 不能解释成“对该函数完成了完整中端优化”。这两个版本用于检查不同编译选项下仍能生成可执行文件；优化现象比较使用的是重新从 `fib.c` 生成的 IR。加入 `-g` 后，目标文件出现 `.debug_info`、`.debug_line` 等调试节，便于将机器地址映射回源位置；调试信息和程序正确性属于不同维度。

（二） 跨架构整合经验

LLVM IR 并非天然完全不依赖目标：目标三元组、数据布局、函数属性和调用约定必须一致。汇编文本也不等于可执行文件：外部函数地址由链接阶段确定，运行库还依赖 C 标准库和启动代码。本实验生成面向 Linux 应用程序的静态 ELF，没有机械照搬讲义中针对另一套工具链的固定链接地址；`-mcmodel=medany` 和 `--no-relax` 均明确记录在构建参数中。

最终的 58 条构建与检查命令都返回 0。日志中 `overriding the module target triple` 表示构建参数将 IR 中的 Linux GNU 标记明确覆盖为 Linux musl；对已经生成的汇编文件传入共同编译参数时还会产生少量“参数未使用”警告。这些诊断全部保留在日志中，但不代表构建失败。

八、 进阶要求：MLIR 与 AscendNPU IR

（一） 多级 IR 与语义逐层下降

LLVM IR 已经采用 SSA、基本块和接近机器的数据操作，适合通用优化及后端代码生成；但卷积、矩阵乘、张量形状等高层语义在下降为 `load/add/store` 后难以恢复。NPU 还具有 Cube/Vector 两类计算核以及 GM、UB、L1 等存储层次，通用 LLVM IR 中没有直接对应的抽象。课程提供的 AscendNPU IR 资料据此说明 MLIR 的设计动机：不同抽象层使用不同方言表达，通过 Lowering 在保持程序语义的前提下逐步引入硬件信息。

MLIR 的基本构成包括 Dialect、Operation 和 Type。Dialect 提供操作与类型的命名空间；Operation 是 IR 的基本节点，可以携带操作数、结果和编译期属性；Type 描述值或内存对象的类型。不同方言可以同时存在于一个模块中，因此一次 Lowering 不必把整个程序直接改写成最终指令，而可以只替换当前阶段已经合法化的操作。

表 7 AscendNPU IR 三个核心方言的职责划分

方言	抽象层次	主要表达内容	代表性操作或属性
HFusion	高层、硬件无关	张量级算子语义，以及算子融合、形状传播和自动调度	`hfusion.vadd`、`hfusion.matmul`
HIVM	NPU 硬件抽象	NPU 核间同步、片上内存分配、流水向量化，以及 GM/UB 地址空间	`hivm.hir.load`、`hivm.hir.vadd`、`hivm.hir.store`
HACC	异构执行抽象	Host/Device 启动语义、设备入口和 tiling 函数绑定	`hacc.entry`、设备函数属性

这三层并不是互相替代的三种写法。HFusion 保留便于融合和调度的算子语义；HIVM 将算子映射到 NPU 存储与计算资源；HACC 说明设备函数如何进入异构运行时。此后才下降到 LLVM IR intrinsic，由后端完成寄存器分配、指令调度与机器码生成 [1]。

（二） VecAdd 的逐层 Lowering 分析

课件以 16 个 i16 元素的向量加法为贯穿示例。前端 Arith/Tensor IR 只表达“两个张量相加”，尚未确定内存位置。arith2fusion 或相应转换首先把值语义的 tensor 转成内存语义的 memref，并用 hfusion.vadd 保存高层向量加法语义。随后 convert-to-hivm-op 将一个高层操作展开为数据搬入、计算和结果写回，同时为中间缓冲标注 UB 地址空间。

Listing 4 VecAdd 从 HFusion 到 HIVM 的结构变化

```
// HFusion: one tensor-level operation
%c = hfusion.vadd ins(%a, %b : memref<16xi16>)
                     outs(%cbuf : memref<16xi16>)

// HIVM: explicit storage and data movement
%ub_a = memref.alloc() : memref<16xi16, #hivm.address_space<ub>>
%ub_b = memref.alloc() : memref<16xi16, #hivm.address_space<ub>>
%ub_c = memref.alloc() : memref<16xi16, #hivm.address_space<ub>>
hivm.hir.load  ins(%a_gm)  outs(%ub_a)
hivm.hir.load  ins(%b_gm)  outs(%ub_b)
hivm.hir.vadd  ins(%ub_a, %ub_b) outs(%ub_c)
hivm.hir.store ins(%ub_c) outs(%c_gm)
```

Lowering 后出现的 gm 与 ub 不是注释，而是后续内存规划和指令选择需要的类型信息。对于该示例，每个向量占 32 字节，三个 UB 缓冲区可依次安排在偏移 0、32 和 64。Plan-Memory 将抽象的 memref.alloc 具体化为不重叠的地址区间；Inject-Sync 在数据搬运与向量计算之间插入等待，避免 vadd 在 load 完成前读取数据。更大的张量还可以通过 Multi-Buffer 让当前分块计算与下一分块搬运重叠，从而隐藏访存延迟。

表 8 VecAdd 下降过程中的信息变化

层级	主要表示	新增或保留的信息
Level 0	Arith/Tensor IR	张量类型和逐元素加法，不包含设备信息
Level 1	HFusion IR	NPU 编译流程可识别的高层算子，便于合法化、化简和融合
Level 2	HIVM IR	GM/UB 地址空间、显式搬运、片上缓冲和同步关系
Level 3	HACC 语义	设备入口、Host/Device 函数类别和启动关系
Level 4	LLVM IR	llvm.npu.* intrinsic，交由 LLVM 后端生成设备二进制

这一过程说明 Lowering 的关键是把隐含约束转成显式 IR。高层的一次加法在 HIVM 中成为两次 GM 到 UB 的搬运、一次向量运算和一次 UB 到 GM 的写回；到了 HACC 层，函数还要带有设备入口属性。每一层仍表达同一计算，但能够执行的分析和优化已经不同。

（三） Pass 流水线、工具链与验证范围

课件给出的完整思路可以概括为：前端方言先经过 Legalize 与 Decompose；HFusion 阶段执行算子化简、融合和自动调度；转入 HIVM 后进行内存规划、同步插入、Mix Kernel 拆分与多缓冲；HACC 绑定异构执行语义，最后由 `hivmc` 转成 LLVM IR 并生成 `kernel.o`。其中 `Split-Mix-Kernel` 对同时使用 Cube 与 Vector 的算子有意义；纯向量 VecAdd 只需映射到 Vector 核，不发生双核拆分。

`bishengir-opt` 可单独运行 Pass，并利用 MLIR 的打印选项观察每一步变化。若具备 CANN 与毕昇编译器环境，可以采用以下命令保存中间 IR：

Listing 5 逐 Pass 观察 AscendNPU IR 的命令形式

```
bishengir-opt add.mlir \
  --convert-hfusion-to-hivm \
  --hivm-plan-memory \
  --hivm-inject-sync \
  --mlir-print-ir-after-all 2>&1 | tee after-each-pass.mlir

bishengir-translate --mlir-to-llvmir add.mlir > add.ll
bishengir-compile add.mlir -enable-hivm-compile -o kernel.o
```

本实验对上述内容完成了课件、官方架构说明和 VecAdd 示例之间的对应分析 [1, 2]。当前实验环境没有安装 CANN，也没有可用的昇腾 NPU 或 Atlas 开发板，因此没有把这些命令的预期输出写成实测结果。基础部分的 570 次执行只统计 RISC-V ELF 在 QEMU 中的真实运行；进阶部分给出可复现的工具链路径和逐层 IR 判读依据，二者的证据范围分别说明。

九、 结论与复现方法

本实验完成了预处理、前端分析、LLVM IR、目标汇编、目标文件及链接后 ELF 的证据链。手写 RISC-V 程序通过课程运行库完成输入输出，与移植后的手写 LLVM IR 以及源程序编译结果在 110 组输入上保持一致。汇编部分同时验证了 32 位整数语义、64 位寄存器环境、调用保存规则、短路副作用和浮点向零截断等容易出错的边界。

Listing 6 实验复现命令

```
python3 scripts/build.py
python3 scripts/test.py
echo 5 | qemu-riscv64 build/fib-asm
printf '7 3\n' | qemu-riscv64 build/features-asm
```

参考文献

[1] AscendNPU IR. 架构设计. https://ascendnpu-ir.gitcode.com/zh_cn/introduction/architecture.html, 2026. 访问日期：2026-10-11.

[2] AscendNPU IR. 编译与执行示例. https://ascendnpu-ir.gitcode.com/zh_cn/sources/introduction/quick_start/examples_zh.html, 2026.

[3] LLVM Project. *Clang Command Guide*, 2026.

[4] LLVM Project. *LLVM Language Reference Manual*, 2026.

[5] RISC-V International. *RISC-V ABIs Specification*, 2026.